Class modifiers

> **Version note:** Class modifiers, besides `abstract`, require a language version of at least 3.0.

Class modifiers control how a class or mixin can be used, both from within its own library, and from outside of the library where it's defined.

Modifier keywords come before a class or mixin declaration. For example, writing `abstract class` defines an abstract class. The full set of modifiers that can appear before a class declaration include:

- `abstract`
- `base`
- `final`
- `interface`
- `sealed`
- `mixin`

Only the `base` modifier can appear before a mixin declaration. The modifiers do not apply to other declarations like `enum`, `typedef`, or `extension`.

When deciding whether to use class modifiers, consider the intended uses of the class, and what behaviors the class needs to be able to rely on.

No modifier

To allow unrestricted permission to construct or subtype from any library, use a `class` or `mixin` declaration without a modifier. By default, you can:

- Construct new instances of a class.
- Extend a class to create a new subtype.
- Implement a class or mixin's interface.
- Mix in a mixin or mixin class.

`abstract`

To define a class that doesn't require a full, concrete implementation of its entire interface, use the `abstract` modifier.

Abstract classes cannot be constructed from any library, whether its own or an outside library. Abstract classes often have abstract methods.

```
// Library a.dart
abstract class Vehicle {
  void moveForward(int meters);
}
```

Google uses cookies to deliver its services, to personalize ads, and to analyze traffic. You can adjust your privacy controls anytime in your Google settings. Learn more.

Okay

```
// Library b.dart
import 'a.dart';

// Error: Cannot be constructed
Vehicle myVehicle = Vehicle();

// Can be extended
class Car extends Vehicle {
  int passengers = 4;
  // ...
}

// Can be implemented
class MockVehicle implements Vehicle {
  @override
  void moveForward(int meters) {
    // ...
  }
}
```

If you want your abstract class to appear to be instantiable, define a factory constructor.

base

To enforce inheritance of a class or mixin's implementation, use the `base` modifier. A base class disallows implementation outside of its own library. This guarantees:

- The base class constructor is called whenever an instance of a subtype of the class is created.
- All implemented private members exist in subtypes.
- A new implemented member in a `base` class does not break subtypes, since all subtypes inherit the new member.
  - This is true unless the subtype already declares a member with the same name and an incompatible signature.

You must mark any class which implements or extends a base class as `base`, `final`, or `sealed`. This prevents outside libraries from breaking the base class guarantees.

```
// Library a.dart
base class Vehicle {
  void moveForward(int meters) {
    // ...
  }
}
```

```
// Library b.dart
import 'a.dart';

// Can be constructed
Vehicle myVehicle = Vehicle();

// Can be extended
base class Car extends Vehicle {
  int passengers = 4;
  // ...
}

// ERROR: Cannot be implemented
base class MockVehicle implements Vehicle {
  @override
  void moveForward() {
```

Google uses cookies to deliver its services, to personalize ads, and to analyze traffic. You can adjust your privacy controls anytime in your Google settings. Learn more.

interface

To define an interface, use the `interface` modifier. Libraries outside of the interface's own defining library can implement the interface, but not extend it. This guarantees:

- When one of the class's instance methods calls another instance method on `this`, it will always invoke a known implementation of the method from the same library.
- Other libraries can't override methods that the interface class's own methods might later call in unexpected ways. This reduces the fragile base class problem.

```
// Library a.dart
interface class Vehicle {
  void moveForward(int meters) {
    // ...
  }
}
```

```
// Library b.dart
import 'a.dart';

// Can be constructed
Vehicle myVehicle = Vehicle();

// ERROR: Cannot be inherited
class Car extends Vehicle {
  int passengers = 4;
  // ...
}

// Can be implemented
class MockVehicle implements Vehicle {
  @override
  void moveForward(int meters) {
    // ...
  }
}
```

abstract interface

The most common use for the `interface` modifier is to define a pure interface. Combine the `interface` and `abstract` modifiers for an `abstract interface class`.

Like an `interface` class, other libraries can implement, but cannot inherit, a pure interface. Like an `abstract` class, a pure interface can have abstract members.

final

To close the type hierarchy, use the `final` modifier. This prevents subtyping from a class outside of the current library. Disallowing both inheritance and implementation prevents subtyping entirely. This guarantees:

- You can safely add incremental changes to the API.
- You can call instance methods knowing that they haven't been overwritten in a third-party subclass.

Final classes can be extended or implemented within the same library. The `final` modifier encompasses the effects of `base`, and therefore any subclasses must also be marked `base`, `final`, or `sealed`.

```
// Library a.dart
final class Vehicle {
```

Google uses cookies to deliver its services, to personalize ads, and to analyze traffic. You can adjust your privacy controls anytime in your Google settings. Learn more.

```
// Library b.dart
import 'a.dart';

// Can be constructed
Vehicle myVehicle = Vehicle();

// ERROR: Cannot be inherited
class Car extends Vehicle {
  int passengers = 4;
  // ...
}

class MockVehicle implements Vehicle {
  // ERROR: Cannot be implemented
  @override
  void moveForward(int meters) {
    // ...
  }
}
```

`sealed`

To create a known, enumerable set of subtypes, use the `sealed` modifier. This allows you to create a switch over those subtypes that is statically ensured to be *exhaustive*.

The `sealed` modifier prevents a class from being extended or implemented outside its own library. Sealed classes are implicitly abstract.

- They cannot be constructed themselves.
- They can have factory constructors.
- They can define constructors for their subclasses to use.

Subclasses of sealed classes are, however, not implicitly abstract.

The compiler is aware of any possible direct subtypes because they can only exist in the same library. This allows the compiler to alert you when a switch does not exhaustively handle all possible subtypes in its cases:

```
sealed class Vehicle {}

class Car extends Vehicle {}

class Truck implements Vehicle {}

class Bicycle extends Vehicle {}

// ERROR: Cannot be instantiated
Vehicle myVehicle = Vehicle();

// Subclasses can be instantiated
Vehicle myCar = Car();

String getVehicleSound(Vehicle vehicle) {
  // ERROR: The switch is missing the Bicycle subtype or a default case.
  return switch (vehicle) {
    Car() => 'vroom',
    Truck() => 'VROOOOMM',
  };
}
```

Google uses cookies to deliver its services, to personalize ads, and to analyze traffic. You can adjust your privacy controls anytime in your Google settings. Learn more.

Combining modifiers

You can combine some modifiers for layered restrictions. A class declaration can be, in order:

1. (Optional) `abstract`, describing whether the class can contain abstract members and prevents instantiation.
2. (Optional) One of `base`, `interface`, `final` or `sealed`, describing restrictions on other libraries subtyping the class.
3. (Optional) `mixin`, describing whether the declaration can be mixed in.
4. The `class` keyword itself.

You can't combine some modifiers because they are contradictory, redundant, or otherwise mutually exclusive:

- `abstract` with `sealed`. A sealed class is always implicitly abstract.
- `interface`, `final` or `sealed` with `mixin`. These access modifiers prevent mixing in.

Google uses cookies to deliver its services, to personalize ads, and to analyze traffic. You can adjust your privacy controls anytime in your Google settings. Learn more.